

Average-Residual Pooling for Audio Token Pruning in Large Audio-Language Models

Anonymous submission to SLT 2026

Abstract—How should local audio tokens be organized during large audio-language model compression? Most existing methods use fixed choices such as averaging, token retention, or token merging. These choices reduce sequence length, but they do not explicitly preserve local acoustic variation. A short audio window may contain a dominant acoustic pattern as well as brief, task-relevant changes. Average pooling captures the dominant pattern but can smooth out these details. Based on this, we propose a token pruning method, Average-Residual Pooling (ARP), which adds a weighted residual term to the local mean. This method preserves the overall content of the window while retaining local variation. ARP is training-free and can be applied to existing audio-language models at inference time. We evaluate ARP and its multiscale variant on Qwen2-Audio and Kimi-Audio. Our evaluation covers audio captioning on AudioCaps and Clotho, and audio reasoning on MMAU. Experimental results show that ARP preserves 96.8% of the full-token performance on average across two models and three datasets with only 75% of the audio tokens; MARP still retains 90.3% of the full-token performance with only 50% of the audio tokens.

Index Terms—Large Audio-Language Models, Audio Token Pruning, Training-Free Compression

I. INTRODUCTION

Large audio-language models (LALMs) have emerged as an important technology for connecting audio perception with language reasoning [1]–[7]. Earlier research in speech or audio processing often depends on task-specific foundation models for speech recognition, audio classification, or spoken language understanding [8]–[16]. In contrast, recent LALMs connect an audio or speech encoder with a large language model, which allows heterogeneous acoustic signals such as speech, environmental sounds, music, and more complex acoustic scenes to be interpreted through natural language prompts. For example, systems such as Qwen2-Audio and Kimi-Audio [2], [3] show that a unified language-model architecture can take audio inputs in different contexts to generate open-ended text responses to describe, interpret, and reason about audio content. However, an important challenge for LALMs is computational efficiency, which is constrained by the ability to process lengthy audio token sequences.

The first challenge comes from the long temporal sequences inherent in audio representations. An audio encoder usually produces a sequence of hidden states, increasing with the audio duration and the temporal resolution of the encoder. Compared with image patches or text tokens, audio tokens are usually presented much denser in time. For short clips, this representation length is usually manageable to LALMs. However, as

the audio duration increases, or when the LALM needs to keep audio context across multiple dialogue turns, the increased number of tokens can substantially increase inference latency and consume a large fraction of the available context budget during the inference [17]–[20]. Moreover, acoustic information is naturally unevenly distributed over time. Even a short audio recording may contain a stationary background and continuous speech along with a shift in timbre, a change in prosody, or the onset of a new sound source. The information that is relevant to downstream audio reasoning is not necessarily the loudest or longest segment; sometimes, it may instead be a short, low-amplitude, but semantically meaningful acoustic change. Thus, audio token pruning is not only about reducing sequence length, but also about preserving information that is relevant to characterize meaningful changes inside audio recordings.

Existing approaches to audio token pruning in LALMs are associated with different trade-offs. The most common average pooling method [21] provides a global summary of the acoustic state in an audio, but it can smooth out information about short events and subtle changes. Moreover, Top- k retention or token pruning [22] can preserve salient tokens, but they may remove the surrounding context needed for audio understanding. Increasingly, similarity-based merging [23] can reduce redundancy among neighboring tokens. However, in real audio scenes, tokens that are close in representation space are not necessarily interchangeable for audio understanding. For example, they may correspond to different stages of an event, such as the moment before and after an onset. Therefore, audio token pruning is not only about which tokens to keep, remove, or merge, but also how to simultaneously preserve the information to characterize both the global acoustic state and the local changes. An effective pruning method is to cover the information of both representations.

In response, we propose Average-Residual Pooling (ARP), a training-free method for inference-time LALM compression. Specifically, given an intermediate sequence of audio tokens from each audio input, ARP first computes an average representation to capture its global acoustic state. It then estimates token-level residuals as deviations from this global average and ranks the most informative residual components for retention. The final compressed representation contains both a global component and the residual components. Importantly, ARP does not require fine-tuning the audio encoder, the adapter, or the language model, and can be directly applied to the inference pipeline of existing LALMs.

We evaluate ARP on existing LALMs, including Qwen2-Audio and Kimi-Audio, under different compression ratios, datasets, and tasks. ARP is under comparison with average pooling, segment-wise pooling, top- k retention, and similarity-based merging. The experiment results show that residual-aware local compression is an effective alternative to plain average pooling. Under a fixed token budget, ARP reduces the number of audio tokens while keeping competitive downstream performance. The advantage is especially clear on both captioning and reasoning tasks, covering local acoustic details capture and global audio understanding.

II. RELATED WORK

A. Large Audio-Language Models

Large audio-language models align audio encoders with large language models so that speech, sound, and music can be interpreted through natural language prompts. Recent systems, including Qwen-Audio (2023), Qwen2-Audio (2024), Kimi-Audio (2025), AudioFlamingo3 (2025), SALMONN (2023/2024), and GAMA (2024), show that a single language-model interface can perform diverse audio understanding and reasoning tasks [1]–[7]. Related speech and audio LLMs, such as AudioPaLM (2023), SpeechGPT (2023), Pengi (2023), and LTU (2023), further connect audio representations with language generation or instruction following [24]–[27].

The evaluation of audio-language models has also expanded rapidly. AIR-Bench (2024) and AudioBench (2024) evaluate audio-centric instruction following and general audio understanding, MMAU (2024) focuses on expert-level audio understanding and reasoning, and Dynamic-SUPERB (2023) studies instruction-based universal speech evaluation [28]–[31]. These works show the growing importance of general audio understanding, but they also expose the high cost of processing long audio-token sequences. Our work studies whether such sequences can be compressed after the audio encoder without training new model components.

B. Audio Token Pruning

Recent work has started to study direct token pruning for audio-language models. This direction asks how to reduce the audio-token sequence passed to an existing LLM decoder while preserving downstream performance. Towards Audio Token Compression in Large Audio Language Models (2025) [21] studies audio-token reduction before the LLM decoder, while LTBM (Local Temporal Bipartite Merging, 2026) proposes locality-aware training-free audio token merging [23]. These works are closest to our setting because they focus on reducing audio tokens rather than training a new audio-language model from scratch.

Existing training-free approaches mainly include fixed pooling, salient-token retention, or similarity-based merging [21]–[23]. Average pooling [21] preserves temporal coverage but can smooth out short acoustic changes. Top- k pruning [22] can keep salient tokens but may remove nearby context. Similarity-based merging, as in LTBM [23], can reduce redundancy

among local tokens, but similar tokens are not always interchangeable near onsets, speaker changes, or timbre transitions. ARP differs from these methods by representing each local window with both an average component and a residual component, aiming to preserve temporal coverage and local acoustic variation at the same time.

III. METHOD

A. Problem Setup

Let x be an input signal. An audio encoder projects x to a sequence of hidden states $H = [h_1, h_2, \dots, h_T]$. Then a projector maps these states to the input space of the language model. We aim to insert a compression operator at this token space, either before the adapter or between the adapter and the language model. For a target token ratio $\rho \in (0, 1]$ which denotes the fraction of audio tokens to be retained, the target number of audio tokens or token budget is: $B = \lceil \rho T \rceil$. Our goal is to construct a compression operator C_ρ :

$$C_\rho(H) = Z, \quad |Z| \leq B. \quad (1)$$

We want to highlight that the audio encoder, its projector, the decoder, and the prompt all remain unchanged in this setting, while the intermediate audio token sequence is compressed into a shorter sequence. The audio encoder, projector, adapter, language model, prompt, and decoding strategy are all kept unchanged. Only the intermediate audio token sequence is replaced by a shorter sequence.

B. Average-Residual Pooling

The pipeline of ARP is shown in Fig. 1. The input audio tokens are initially segmented into local windows. For each window, ARP computes an average token and token-wise residuals. We then propose a residual gate to select which local residual information to keep. The details of each component are described as follows.

Local windows. Since audio tokens have different temporal structures across time, ARP uses local windows as the basic compression unit. Given the audio token sequence H , we segment it into K consecutive windows, where $\mathcal{W}_j$ denotes the continuous token sequence in the j -th window. Specifically, $\mathcal{W}_j = \{h_{s_j}, h_{s_j+1}, \dots, h_{e_j}\}$. This windowing design preserves the temporal structure of the audio while ensuring the ability to capture both stationary patterns and subtle changes.

Average token in a local window. For the j -th window $\mathcal{W}_j$, ARP first computes the average token μ_j simply by averaging the token sequences in that window. This average token μ_j represents the stationary acoustic state in the window. This can be a sustained timbre, a stable speaker state, or the environmental soundscape. However, as we discussed earlier, a single average token may smooth out short sound events, sound onsets, and subtle acoustic changes.

Token residuals in a local window. Given the local average token, ARP computes the residual of each token around this local average by $r_i = h_i - \mu_j$, where $h_i \in \mathcal{W}_j$. The residual r_i describes the direction and magnitude by which token h_i

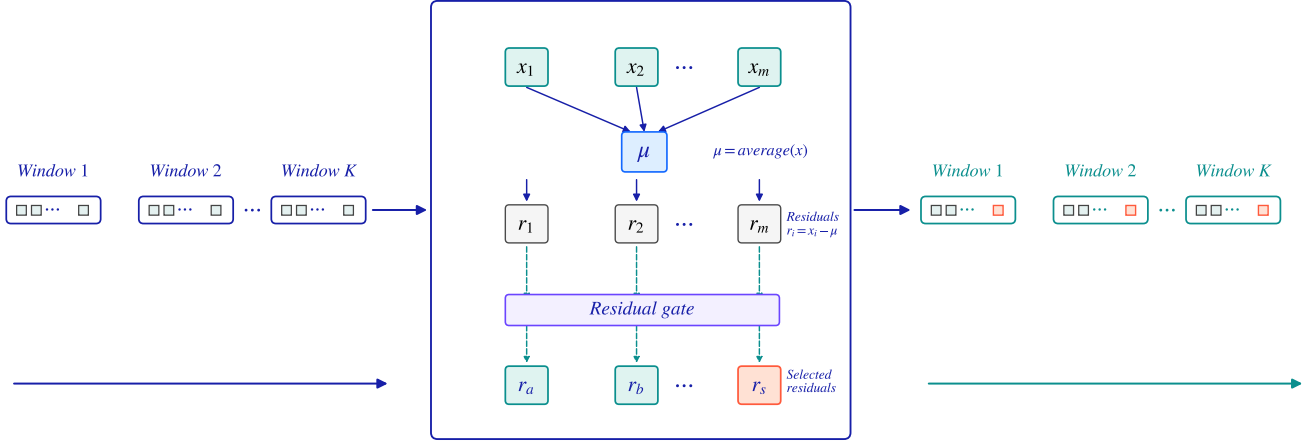


Fig. 1. Overview of Average-Residual Pooling (ARP).

deviates from the stationary state of the window. A small residual indicates that the token is close to the local average, suggesting a relatively stationary acoustic region. In contrast, a large residual token may indicate an event onset, a speaker change, a timbre change, or other subtle acoustic variation.

Residual gate for token selection. The residual gate is a critical part of ARP. Its role is to decide which local changes should be kept under the token budget. For each residual, we first compute a token-level residual score: $s_i = \|r_i\|_2$. A larger s_i means that the token is farther from the local average. It may therefore contain more local variation. We also compute a window-level variation score:

$$v_j = \frac{1}{|\mathcal{W}_j|} \sum_{h_i \in \mathcal{W}_j} \frac{\|h_i - \mu_j\|_2}{\|\mu_j\|_2 + \epsilon}. \quad (2)$$

When v_j is small, it implies that the local window includes more stationary information, and the average token is sufficient to characterize the audio recording. On the other hand, when v_j is large, the window contains stronger local variation. In this case, residual candidates from this window receive higher ranking scores, making them more likely to be selected under the residual budget.

Budget control. In our design, the ARP must satisfy the total token budget B . Since each window keeps one average token, the remaining residual budget is $R = B - K$. If $R \leq 0$, ARP reduces to segment-wise average pooling. If $R > 0$, ARP uses the residual gate to select additional tokens. For a candidate token $h_i \in \mathcal{W}_j$, we combine its token-level residual score and the variation score of its window as $q_i = s_i \cdot v_j$. We then keep the top R candidates with the highest q_i scores and assign them back to their corresponding windows. This ensures that the final output length does not exceed B , while directing the residual budget toward windows with greater local variations.

Compressed output. Let $\mathcal{S}_j$ be the set of token indices selected by the residual gate in window $\mathcal{W}_j$. The output of this window is

$$Z_j = [\mu_j; \{h_i : i \in \mathcal{S}_j\}]. \quad (3)$$

Here, μ_j keeps the stationary acoustic state in each local window, while the selected tokens are indicative of changes in local windows. Since $h_i = \mu_j + r_i$, keeping these selected original tokens is equivalent to keeping the corresponding residual information in the original token space. The ultimate compressed sequence is obtained by concatenating the selected outputs from all windows $Z = [Z_1, Z_2, \dots, Z_K]$. For local windows containing relatively stationary information, ARP behaves close to average pooling, whereas for windows with greater local variations, it attempts to retain meaningful residual information. Our proposed ARP reduces the chance that fine-grained audio changes are smoothed out by global average pooling.

C. Variant: Multiscale ARP

We further design a multiscale variant of ARP, called Multiscale Average-Residual Pooling (MARP), which replaces the uniform temporal segmentation with windows of varying sizes. The motivation is that audio signals are often non-uniform over time. Some parts are relatively stable, while others contain rapid acoustic changes. A single uniform partition may therefore allocate tokens inefficiently, particularly for longer audio clips. MARP first estimates local acoustic change from adjacent audio tokens. Given the hidden states $\{h_i\}_{i=1}^T$, we compute a local change score between neighboring tokens:

$$d_i = \|h_{i+1} - h_i\|_2, \quad i = 1, \dots, T-1. \quad (4)$$

We use a fixed number of temporal regions, denoted by M . In all experiments, we set $M = 4$. MARP selects the top $M-1$ positions with the largest adjacent change scores as region boundaries. No threshold is used. The selected boundary positions are sorted in temporal order and split the original sequence into M consecutive regions:

$$\mathcal{R}_m = \{h_{b_{m-1}}, \dots, h_{b_m-1}\}, \quad m = 1, \dots, M. \quad (5)$$

Here, $b_0 = 1$ and $b_M = T + 1$. If the target token budget is smaller than the number of regions, we reduce the number of

regions so that each region receives at least one compressed token.

The total target budget is $B = \lceil \rho T \rceil$. We allocate this budget to regions according to their temporal lengths. Let $|\mathcal{R}_m|$ denote the number of tokens in region m . The initial regional budget is computed as

$$\tilde{B}_m = B \cdot \frac{|\mathcal{R}_m|}{T}. \quad (6)$$

We take the integer part of each $\tilde{B}_m$ and ensure that every region keeps at least one token. Any remaining budget is assigned to regions with the largest fractional remainders until the total number of compressed tokens is exactly B . If the sum exceeds B , we remove tokens from regions with larger budgets and keep at least one token per region, which gives a regional budget B_m satisfying $\sum_m B_m = B$.

Within each region, MARP applies the same average-residual structure as ARP. Each region is divided into local windows. For each window, MARP computes a local mean token and token-wise residuals around this mean. The residual score is the L2 norm of the residual in the LLM input embedding space. The selected residual slot is represented by the corresponding original token h_i , which is equivalent to preserving the residual information $r_i = h_i - \mu_j$ in the original token space. Thus, MARP uses region-oriented temporal partitioning before applying the same residual-aware selection rule as ARP.

IV. EXPERIMENTAL SETUP

A. Models and Datasets.

We evaluate on two large audio-language models: Qwen2-Audio and Kimi-Audio [2], [3]. All model weights are kept without fine-tuning. Compression is applied only to the intermediate audio token sequence at inference time. We use AudioCaps [32] and Clotho [33] for audio captioning, and MMAU [30] for audio reasoning. The evaluation splits contain 883 examples from AudioCaps, 1,043 examples from Clotho, and 1,000 examples from MMAU. For AudioCaps and Clotho, we use the prompt: “Generate a caption in English.” For MMAU, we use the question and answer choices in the following format: “{question} Options: (A) {choice_A} (B) {choice_B} ... Answer with exactly one option letter in parentheses.”

B. Compression Setup

Compression position. All experiments use LLM-side audio-token compression. Specifically, we first let each model’s original audio front-end and adapter/projector produce the audio-token embeddings in the language-model input space. We then replace only the audio-token span in the LLM input embeddings with a shorter compressed sequence. Text tokens, prompts, model weights, and decoding settings are kept unchanged.

Compression settings. We evaluate the setting without compression as the uncompressed baseline and use target token ratios $\rho \in \{0.75, 0.50, 0.25\}$. For all compression methods,

the target audio-token budget is $B = \lceil \rho T \rceil$, where T is the original number of LLM-side audio tokens. Compression is applied to the audio-token embeddings after the audio encoder and projector have mapped them into the language model’s input space. It is worth noting that the proposed compression operates exclusively on the audio-token representations, while the text tokens, prompt structure, model parameters, and decoding configuration remain entirely unmodified.

Baselines, ARP, and MARP. We compare ARP and MARP with three broader types of training-free baselines: average-based pooling, token-retention, and token-merging. UniAvg [21] splits the audio-token sequence into B segments and averages each segment. GlobalTopK keeps the top- B tokens according to their L2 norm, while SegTopK [22] first splits the sequence into B segments and keeps the highest-scoring token from each segment. LocalMerge is a local similarity-based merging method inspired by LTBM [23]. It uses cosine similarity as the matching score and restricts candidate matches to a local temporal window of size $w = 8$. We use it as a representative merge-based baseline rather than an exact reproduction of the original LTBM pipeline.

For ARP, the local window size is fixed to $w = 8$, the same as other related baselines. For MARP, we use $M = 4$ temporal regions and the same local window size $w = 8$. MARP selects the top $M - 1$ adjacent token-change positions as region boundaries, without using a threshold.

Decoding details. We use deterministic decoding for all methods. The maximum generation length is 256 new tokens for AudioCaps and Clotho, and 16 new tokens for MMAU. The batch size is 4. And we assess stability by paired bootstrap over evaluation samples rather than by random-seed variance.

V. RESULTS

A. Main Compression Results

Table I reports the main compression results on two audio-language models, three downstream datasets, and three target token ratios. AudioCaps and Clotho are audio captioning tasks, so we use CIDEr-D. MMAU is an audio understanding and reasoning task, so we use accuracy. Baseline without compression denotes the inference without audio token compression, which serves as the uncompressed reference.

In general, our proposed ARP approach shows the most consistent performance across 18 different compression settings with 2 models, 3 datasets, and 3 target token ratios. Compared with other compression baselines, ARP and MARP together achieve the best results in 7 settings. They also appear in the top two 15 times. In 11 out of 18 settings, at least one ARP-family method is among the top two. For both Kimi-Audio and Qwen2-Audio, the ARP family achieves the best result on every dataset at least once under some compression ratio. More specifically, ARP has an average rank of **2.44**, making it the most stable single method. Among all baselines, LocalMerge achieves the best performance, with an average rank of around 2.56, though its gains are concentrated in low-budget settings. More importantly, MARP does not

TABLE I

MAIN RESULTS ACROSS MODELS, DATASETS, AND TARGET TOKEN RATIOS. AUDIOCAPS AND CLOTHO ARE EVALUATED BY CIDER-D, AND MMAU IS EVALUATED BY ACCURACY. THE W/O COMPRESSION ROW REPORTS THE UNCOMPRESSED REFERENCE AND IS REPEATED UNDER EACH KEEP-RATIO COLUMN FOR COMPARISON. AMONG ALL METHODS, THE BEST RESULT IN EACH COLUMN IS SHOWN IN **BOLD**, AND THE SECOND-BEST RESULT IS UNDERLINED.

Model	Type	Method	AudioCaps			Clotho			MMAU		
			0.75	0.50	0.25	0.75	0.50	0.25	0.75	0.50	0.25
Kimi-Audio-7B	Baseline	w/o Compression	0.4385	0.4385	0.4385	0.3276	0.3276	0.3276	0.7050	0.7050	0.7050
		UniAvg	0.3950	0.3584	0.3135	0.3113	0.2368	0.1852	0.7020	<u>0.6900</u>	<u>0.6680</u>
		GlobalTopK	0.4095	<u>0.3689</u>	0.3102	0.3077	0.2391	0.1767	0.7060	<u>0.6900</u>	0.6700
		SegTopK	0.4021	0.3682	0.3101	0.3095	0.2418	<u>0.1842</u>	0.7010	<u>0.6900</u>	0.6640
		LocalMerge	<u>0.4097</u>	0.3737	<u>0.3143</u>	0.3136	0.2405	0.1783	0.7030	0.6890	0.6700
	ARP Family	ARP	0.4103	0.3685	0.3157	0.3124	<u>0.2442</u>	0.1791	<u>0.7050</u>	0.6900	0.6640
		MARP	0.4079	0.3668	0.3107	0.3091	0.2490	0.1791	0.7020	0.6930	0.6650
	Qwen2-Audio-7B	w/o Compression	0.5182	0.5182	0.5182	0.3194	0.3194	0.3194	0.5900	0.5900	0.5900
		UniAvg	0.4917	0.4940	0.4337	0.2939	0.2998	<u>0.2497</u>	<u>0.5850</u>	0.5520	0.4700
		GlobalTopK	0.4319	0.2772	0.0769	0.2622	0.1277	0.0344	0.5270	0.4450	0.3710
		SegTopK	0.5076	0.5106	0.3898	0.2931	<u>0.2966</u>	0.2319	0.5720	0.5400	0.4580
		LocalMerge	0.4970	0.4959	<u>0.4206</u>	0.2990	0.2952	0.2752	0.5770	<u>0.5550</u>	0.5030
	ARP Family	ARP	<u>0.5037</u>	0.5080	0.4096	<u>0.3031</u>	0.2868	0.2458	0.5890	<u>0.5560</u>	0.4690
		MARP	0.5015	<u>0.5097</u>	0.4074	0.3062	0.2911	0.2427	0.5810	0.5580	0.4670

always yield the best performance, but it demonstrates strong performance in several long-audio understanding and audio reasoning scenarios, such as Kimi-Audio on Clotho@0.50 and MMAU@0.50, and Qwen2-Audio on Clotho@0.75 and MMAU@0.50.

The results also suggest that different compression methods have different strengths. TopK methods can keep important local tokens, but they are unstable under strong compression and on long audio. Although ARP also uses an L2-based score, it ranks residuals relative to a local mean rather than raw token norms over the full sequence. Therefore, ARP measures local deviation while maintaining temporal coverage. Average pooling gives good coverage, but it can smooth local variation. LocalMerge has strong coverage at low keep ratios, but it does not dominate all tasks. In contrast, ARP and MARP aim to balance coverage and variation preservation. A representative example is MMAU@0.75. On Kimi-Audio, ARP keeps the accuracy unchanged, from 0.7050 without compression to 0.7050 with 25% fewer audio tokens. Similarly, on MMAU with Qwen2-Audio, ARP drops only from 0.5900 to 0.5890, an absolute decrease of just 0.0010, when discarding 25% of the tokens.

B. Efficiency Analysis

For each compression result, we compute performance retention as the compressed score divided by the full-token score on the same model and dataset. A higher retention value means that the method preserves more task performance under the same audio-token budget. Figure 2 shows that ARP-family methods make competitive and stable use of the remaining audio-token budget. At a target token ratio of 0.75, ARP and MARP keep a larger proportion of the full-token performance

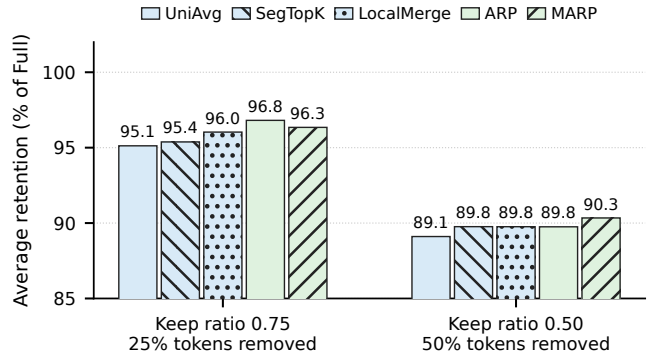


Fig. 2. Budgeted performance efficiency under fixed audio-token budgets. We measure efficiency as average performance retention, defined as the compressed score divided by the full-token score. Results are averaged over 6 model-dataset settings. Green bars represent ARP-family.

than the baseline methods. At the target token ratio of 0.5, MARP achieves the highest average retention, showing that multiscale residual pooling remains effective when the audio-token budget becomes tighter.

Furthermore, we report peak GPU memory in Table II. This experiment evaluates memory usage on Qwen2-Audio under the same batch size and decoding settings. Compared with the full-token model, ARP and MARP consistently reduce peak memory across all tested target token ratios. On Clotho, peak memory decreases from 24.159G with the full-token model to 23.519G at a 0.75 target token ratio and 23.309G at 0.50 target token ratio. On MMAU, peak memory decreases from 23.827G to 23.517G at 0.75 target token ratio and

TABLE II

PEAK GPU MEMORY ON QWEN2-AUDIO. MEMORY IS MEASURED DURING GENERATION UNDER THE SAME BATCH SIZE AND DECODING SETTINGS.

Method	Target Token Ratio	Clotho	MMAU
Full	1.00	24.159G	23.827G
ARP	0.75	23.519G	23.517G
ARP	0.50	23.309G	23.467G
MARP	0.75	23.517G	23.517G
MARP	0.50	23.309G	23.467G

TABLE III

WINDOW-SIZE SENSITIVITY ON QWEN2-AUDIO. CLOTHO USES CIDER-D, AND MMAU USES ACCURACY. BEST RESULTS IN EACH METHOD-DATASET-RATIO GROUP ARE BOLDED.

Method	Ratio	Clotho				MMAU			
		$k = 4$	$k = 8$	$k = 16$	$k = 32$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
ARP	0.50	0.2960	0.2948	0.3034	0.3039	0.5430	0.5280	0.5310	0.5370
MARP	0.50	0.2917	0.3017	0.3147	0.3101	0.5490	0.5350	0.5360	0.5380
ARP	0.75	0.2921	0.3046	0.3089	0.3032	0.5750	0.5600	0.5610	0.5620
MARP	0.75	0.2927	0.3095	0.3072	0.3026	0.5770	0.5650	0.5650	0.5580

23.467G at 0.50 target token ratio. These results show that the LLM-side audio token compression can reduce the memory footprint of audio-conditioned generation while preserving strong downstream performance.

C. Window-Size Sensitivity

This section discusses the sensitivity of ARP-family methods to the local window size. We use Qwen2-Audio to evaluate ARP and MARP on Clotho and MMAU under two ratios, 0.75 and 0.50. We vary the local window size $k \in \{4, 8, 16, 32\}$ while keeping the model weights, prompts, decoding settings, and evaluation splits unchanged.

Table III shows that the best window size is task-dependent. On Clotho, which is a captioning task with longer audio clips, larger windows are often helpful. For example, MARP obtains the best score at $k = 16$ under 0.50 target token ratio, and ARP obtains the best score at $k = 16$ under 0.75 target token ratio.

MMAU shows a different pattern. For both ARP and MARP, the best results are achieved at $k = 4$ under both target token ratios. Table III shows that the best window size can vary across tasks. Clotho sometimes benefits from larger windows, while MMAU tends to prefer smaller windows. However, we do not tune the window size separately for each dataset in the main experiments. Instead, we use a fixed value $w = 8$ for all models, datasets, and target token ratios. The sensitivity study is included only to show that ARP-family methods remain competitive across a reasonable range of local granularities.

D. Ablation Study

We further conduct experiments on which component of the ARP family contributes to the final performance. Table IV reports an ablation study on Qwen2-Audio-7B at a fixed 0.75 ratio. We compare our proposed ARP and MARP with three controlled variants. *ARP w/o Residual* removes the residual

TABLE IV

ABLATION STUDY ON QWEN2-AUDIO-7B AT 0.75 RATIO. AUDIOCAPS AND CLOTHO USE CIDER-D, AND MMAU USES ACCURACY. ARP w/o RESIDUAL IS EQUIVALENT TO UNIAVG.

Method	Variant	AudioCaps	Clotho	MMAU
ARP w/o Residual	mean only	0.4917	0.2939	0.5850
MARP w/o Residual	multiscale mean only	0.4965	0.3052	0.5850
ARP w/ Random Residual	random residual anchor	0.4951	0.2936	0.5880
ARP	ours	0.5037	0.3031	0.5890
MARP	ours	0.5015	0.3062	0.5810

term from ARP. This method is actually equivalent to uniform average pooling, i.e., UniAvg. *MARP w/o Residual* still keeps the multiscale region partition of MARP, but it removes the residual correction. *ARP w/ Random Residual* keeps the ARP form, but it uses a random residual anchor instead of an informative one.

The ablation results clearly show that at this compression rate, all residual components are important for ARP-family compression. Removing the residual term from ARP gives ARP w/o Residual, which is the same as UniAvg. This variant is consistently weaker than ARP on all three datasets. For example, ARP improves AudioCaps from 0.4917 to 0.5037 and improves MMAU from 0.5850 to 0.5890. This suggests that average pooling alone provides temporal coverage, but it can smooth out useful local acoustic variation.

The multiscale design also matters. MARP w/o Residual improves over ARP w/o Residual on AudioCaps and Clotho, showing that region-aware pooling can help longer and more diverse audio inputs. However, the full MARP still gives the best Clotho result, improving from 0.3052 to 0.3062 after adding the residual term. This indicates that multiscale coverage and residual preservation are complementary.

Finally, ARP w/ Random Residual does not match the full ARP. Although it keeps the residual form, replacing the informative residual anchor with a random one reduces performance on AudioCaps and Clotho. This shows that selecting an informative residual direction is also important for preserving local acoustic changes under compression.

VI. CONCLUSION

We study training-free audio-token compression for large audio-language models and propose Average-Residual Pooling (ARP) and its multiscale variant MARP. Across two models, three datasets, and three target token ratios, ARP family methods show stable performance under limited token budgets. Among 18 compressed settings, ARP and MARP achieve 7 best results, and rank in the top two in 11 settings when considered as a family. ARP obtains the best average rank among all single methods. The results suggest that audio-token compression should not only focus on how to reduce sequence length. We also need to switch our attention to preserving local acoustic variation while maintaining temporal coverage. Future work will focus on the mechanism behind residual scoring, including how different norm choices and embedding-space distances affect the selection of local variations.

REFERENCES

- [1] Y. Chu, J. Xu, X. Zhou, Q. Yang, S. Zhang, Z. Yan, C. Zhou, and J. Zhou, “Qwen-Audio: Advancing universal audio understanding via unified large-scale audio-language models,” arXiv preprint arXiv:2311.07919, 2023.
- [2] Y. Chu, J. Xu, Q. Yang, H. Wei, X. Wei, Z. Guo, Y. Leng, Y. Lv, J. He, J. Lin, C. Zhou, and J. Zhou, “Qwen2-Audio Technical Report,” arXiv preprint arXiv:2407.10759, 2024.
- [3] Kimi Team, D. Ding, Z. Ju, Y. Leng, S. Liu, T. Liu, Z. Shang, K. Shen, W. Song, X. Tan, H. Tang, Z. Wang, C. Wei, Y. Xin, X. Xu, J. Yu, Y. Zhang, X. Zhou, and others, “Kimi-Audio Technical Report,” arXiv preprint arXiv:2504.18425, 2025.
- [4] Z. Kong, A. Goel, R. Badlani, W. Ping, R. Valle, and B. Catanzaro, “Audio Flamingo: A novel audio language model with few-shot learning and dialogue abilities,” in *Proc. ICML*, 2024, pp. 25125–25148.
- [5] A. Goel, S. Ghosh, J. Kim, S. Kumar, Z. Kong, S.-g. Lee, C.-H. H. Yang, R. Duraiswami, D. Manocha, R. Valle, and B. Catanzaro, “Audio Flamingo 3: Advancing audio intelligence with fully open large audio language models,” arXiv preprint arXiv:2507.08128, 2025.
- [6] C. Tang, W. Yu, G. Sun, X. Chen, T. Tan, W. Li, L. Lu, Z. Ma, and C. Zhang, “SALMONN: Towards generic hearing abilities for large language models,” in *Proc. ICLR*, 2024.
- [7] S. Ghosh, S. Kumar, A. Seth, C. K. R. Evuru, U. Tyagi, S. Sakshi, O. Nieto, R. Duraiswami, and D. Manocha, “GAMA: A large audio-language model with advanced audio understanding and complex reasoning abilities,” in *Proc. EMNLP*, 2024.
- [8] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in *Proc. NeurIPS*, 2020.
- [9] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “HuBERT: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Trans. Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.
- [10] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, and others, “WavLM: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [11] J. Ao, R. Wang, L. Zhou, C. Wang, S. Ren, Y. Wu, S. Liu, T. Ko, Q. Li, Y. Zhang, Z. Wei, Y. Qian, J. Li, and F. Wei, “SpeechT5: Unified-modal encoder-decoder pre-training for spoken language processing,” in *Proc. ACL*, 2022, pp. 5723–5738.
- [12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, 2023.
- [13] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, and F. Wei, “BEATs: Audio pre-training with acoustic tokenizers,” in *Proc. ICML*, 2023.
- [14] Y. Gong, Y.-A. Chung, and J. Glass, “AST: Audio Spectrogram Transformer,” in *Proc. Interspeech*, 2021, pp. 571–575.
- [15] B. Elizalde, S. Deshmukh, M. A. Ismail, and H. Wang, “Natural language supervision for general-purpose audio representations,” in *Proc. ICASSP*, 2023.
- [16] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio Set: An ontology and human-labeled dataset for audio events,” in *Proc. ICASSP*, 2017, pp. 776–780.
- [17] P. He, Z. Wen, Y. Wang, Y. Wang, X. Liu, J. Huang, Z. Lei, Z. Gu, X. Jin, J. Yang, K. Li, Z. Liu, W. Li, C. Wang, C. He, and L. Zhang, “AudioMarathon: A comprehensive benchmark for long-context audio understanding and efficiency in audio LLMs,” arXiv preprint arXiv:2510.07293, 2025.
- [18] O. Ahia, M. Bartelds, K. Ahuja, H. Gonen, V. Hofmann, S. Arora, S. S. Li, V. Puttagunta, M. Adeyemi, C. Buchireddy, B. Walls, N. Bennett, S. Watanabe, N. A. Smith, Y. Tsvetkov, and S. Kumar, “BLAB: Brutally long audio bench,” arXiv preprint arXiv:2505.03054, 2025.
- [19] K. Luo, L. Lin, Y. Zhang, M. Aloqaily, D. Wang, Z. Zhou, J. Zhang, K. Wang, L. Sun, and Q. Wen, “ChronosAudio: A comprehensive long-audio benchmark for evaluating audio-large language models,” arXiv preprint arXiv:2601.04876, 2026.
- [20] J. Ye, D. Wang, Y. Ye, S. Zhou, W. Lin, M. Han, K. Wang, Z. Yuan, B. Li, H. Shi, J. Shu, J. Song, and B. Zheng, “VoiceGiraffe: A benchmark for extreme long-context audio-language understanding,” arXiv preprint arXiv:2605.27976, 2026.
- [21] S. Bhati, S. Thomas, H. Kuehne, R. Feris, and J. Glass, “Towards audio token compression in large audio language models,” arXiv preprint arXiv:2511.20973, 2025.
- [22] M. Gibier, R. Duroselle, P. Serrano, O. Boeffard, and J.-F. Bonastre, “Segmentwise pruning in audio-language models,” arXiv preprint arXiv:2511.14293, 2025.
- [23] J. Luo, X. Liang, and H. Hu, “Locality matters for training-free audio token compression in audio-language models,” arXiv preprint arXiv:2605.25179, 2026.
- [24] P. K. Rubenstein, C. Asawaroengchai, D. D. Nguyen, A. Bapna, Z. Borsos, F. de Chaumont Quirry, P. Chen, D. El Badawy, W. Han, E. Kharitonov, and others, “AudioPaLM: A large language model that can speak and listen,” arXiv preprint arXiv:2306.12925, 2023.
- [25] D. Zhang, S. Li, X. Zhang, J. Zhan, P. Wang, Y. Zhou, and X. Qiu, “SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities,” in *Findings of EMNLP*, 2023.
- [26] S. Deshmukh, B. Elizalde, R. Singh, and H. Wang, “Pengi: An audio language model for audio tasks,” in *Proc. NeurIPS*, 2023.
- [27] Y. Gong, H. Luo, A. H. Liu, L. Karlinsky, and J. Glass, “Listen, Think, and Understand,” in *Proc. ICLR*, 2024.
- [28] Q. Yang, J. Xu, W. Liu, Y. Chu, Z. Jiang, X. Zhou, Y. Leng, Y. Lv, Z. Zhao, C. Zhou, and J. Zhou, “AIR-Bench: Benchmarking large audio-language models via generative comprehension,” in *Proc. ACL*, 2024.
- [29] B. Wang, X. Zou, G. Lin, S. Sun, Z. Liu, W. Zhang, Z. Liu, A. T. Aw, and N. F. Chen, “AudioBench: A universal benchmark for audio large language models,” in *Proc. NAACL-HLT*, 2025, pp. 4297–4316.
- [30] S. Sakshi, U. Tyagi, S. Kumar, A. Seth, R. Selvakumar, O. Nieto, R. Duraiswami, S. Ghosh, and D. Manocha, “MMAU: A massive multi-task audio understanding and reasoning benchmark,” in *Proc. ICLR*, 2025.
- [31] C.-y. Huang, K.-H. Lu, S.-H. Wang, C.-Y. Hsiao, C.-Y. Kuan, H. Wu, S. Arora, K.-W. Chang, J. Shi, Y. Peng, R. Sharma, S. Watanabe, B. Ramakrishnan, S. Shehata, and H.-y. Lee, “Dynamic-SUPERB: Towards a dynamic, collaborative, and comprehensive instruction-tuning benchmark for speech,” arXiv preprint arXiv:2309.09510, 2023.
- [32] C. D. Kim, B. Kim, H. Lee, and G. Kim, “AudioCaps: Generating captions for audios in the wild,” in *Proc. NAACL-HLT*, 2019, pp. 119–132.
- [33] K. Drossos, S. Lipping, and T. Virtanen, “Clotho: An audio captioning dataset,” in *Proc. ICASSP*, 2020, pp. 736–740.